

Distributed Frame Rendering: Byte-Identical Across Machines, and What a Slow Node Is Worth

Matias Saavedra

Sunday 12th July, 2026

Binary: branch feat/frame-distribution, commits aae49fb (tag binary-frame-dist: the DIST_* mechanism) and 52c40c4 (make GPU=0 + heterogeneous hosts), built atop main 8acde54 (the binary-v7-gpu-periodicity lineage). Node B (*ivy*) runs the GPU=0 build of the same commits. Raw data: experiments/25-frame-distribution/.

Resumen

This report adds the first multi-machine axis to the renderer: frames of the interpolated sequence are distributed *across* nodes by a static cyclic/block-cyclic predicate (three environment variables, one line in the frame loop), while the region-level work queue stays intra-node and untouched. Correctness is settled first: on the production spec (100 frames, 1920×1080 , `diffT` = 0.1), every executor-pinned check — three simulated local nodes in CPU-only cyclic, GPU-only cyclic, and CPU-only block-cyclic(8), plus a real two-machine run over SSH — reproduces the single-process output **100/100 frames byte-identical**, including across compilers and microarchitectures (g++-15/Coffee Lake vs. g++ 16/Haswell). The benchmark is the honest half: against a laptop-alone baseline of 18.75 ± 0.34 s, *every* two-node static configuration loses — best 28.24 ± 0.58 s (+51%), worst 65.12 s — because the second node is $4.56\times$ slower per frame and equal shares strand the wall on it. The ceiling is analytic: a perfectly weighted split would reach only 15.4 s ($1.22\times$), and the measured 3.4–4.2 s ship/collect overhead consumes most of that. Static distribution needs near-homogeneous nodes; a slow node buys capacity, not latency.

1. Motivation: the partition boundary is where prediction fails

Reports 12–23 exhausted most of the single-machine levers: the wall fell from ~ 53 s to 23.7 s through affinity and work reduction, and the CPU pool binds at $<1.4\%$ thread spread in every regime (`13-zoom-points.tex`). The next multiplier is more machines — but *not* by distributing the region queue. Regions are millisecond-scale tasks (median ~ 10 ms post-periodicity), so network dispatch latency would be a first-order cost, and region cost is unknowable before sampling — that unpredictability is the entire reason the intra-node scheduler is dynamic. Any static partition below the frame level inherits the misprediction problem the queue exists to solve.

The frame is therefore the unit of distribution: the two-level hierarchy assigns frames across nodes statically and leaves within-frame scheduling dynamic. What makes the static assignment non-trivial is the $32\times$ per-frame cost spread measured

in `13-zoom-points.tex` (2.37–76.3 s), trending upward along the zoom. A contiguous block assignment strands whichever node draws the deep block; cyclic assignment (node i renders frames $i, i+N, i+2N, \dots$) hands every node a representative cheap/expensive mix. Block vs. cyclic vs. block-cyclic is exactly the static-LB study report 13 earmarked; `S5` delivers its first measured pass — on deliberately heterogeneous hardware.

2. The change: one predicate, three environment variables

Commit `aae49fb` adds `DIST_NODES` / `DIST_RANK` / `DIST_BLOCK` (environment variables; unset = off, byte-for-byte the old behaviour). The entire distribution decision is one line in `main.cpp`’s frame-construction loop:

Código 1: `main.cpp` — the frame-ownership predicate (commit `aae49fb`, abridged).

```

1 bool owned = (distNodes <= 0) || ((i / distBlock) % distNodes == distRank);
2 if (owned)
3 {
4     // fname keeps the GLOBAL frame index i, so the union of all
5     // nodes' outputs is collision-free with no renumbering.
6     sprintf (fname, "%s%04i.png", imageFilePrefix, i);
7     fr[i] = new MandelFrame (uX, uY, lX, lY, resolutionX, resolutionY, fname, iter);
8     fr[i]->frameIndex = i;
9     workQ.append (new MandelRegion (uX, uY, lX, lY, 0, 0,
10                                     resolutionX, resolutionY, fr[i]));
11 }
12 else
13     fr[i] = NULL;    // skipped frame: no buffer, no root region
14 // The interpolation ACCUMULATES (uX += sx1, ...): it must advance
15 // for every i, owned or not, so frame i's corners stay bit-identical
16 // to a single-node run (base + i*step would round differently).
17 uX += sx1; uY += sy1; lX += sx2; lY += sy2; iter += iterInc;

```

Three properties carry all the correctness weight. **(a) The interpolation accumulation is untouched.** Frame i ’s corners are i sequential FP additions; in IEEE 754, $(a+s)+s \neq a+2s$ in the last ulp, so computing corners as `base + i*step` — or running the loop only over owned indices — would seed every pixel’s orbit differently. Ownership gates only the allocation and the enqueue. **(b) Filenames carry the global frame index**, so multi-node collection is a plain file union: ownership is a partition (each i satisfies the predicate for exactly one rank), hence no collisions and no renumbering; a gap in the union is immediately visible. **(c) Nothing downstream knows the feature exists.** The workers drain whatever roots were enqueued; each frame saves itself when its own atomic counter reaches zero; the colormap seed and `CUDAmemSetup` depend only on the spec endpoints and resolution. A side effect: per-node frame-buffer memory drops to owned frames only ($\sim 1/N$ of the ~ 830 MB full sequence).

`DIST_BLOCK` makes the predicate cover the whole static-assignment family with one knob: `block = 1` is pure cyclic; `block =  $k$` is block-cyclic; `block  $\geq$   $\lceil \text{frames}/N \rceil$`

degenerates to contiguous block.

The manager-worker seam (deliberately not built).

Static assignment cannot absorb heterogeneous node speeds or a mispredicted monster frame; the agreed future fix is a dispenser — finish a frame, request the next index, one message per frame. The predicate is the seam: it is the only place that decides whether this process renders frame i . `WorkQueue::append` is already mutex-safe for incremental insertion; the one obligation a dispenser inherits is that `CalcThr::run` exits on a momentarily empty queue, so the next root must be enqueued before the current frame drains. $\mathcal{S}5.4$ quantifies exactly how much a dispenser would be worth on the pair measured here.

Orchestration.

`scripts/dist_frames.sh` drives one job per rank with GNU parallel over SSH: ship the (single, shared) spec, run in a per-rank working directory with the `DIST_*` variables set inline, collect the PNGs back by their global names. Hosts-file lines are `host [rdir [args...]]`, so each node carries its own project root and run configuration — the heterogeneity hook used below. The run `cds` into the working directory rather than encoding node paths into the image prefix, side-stepping the known `char[42]` prefix overflow (`21-periodicity-check.tex`).

The CPU-only build (commit 52c40c4).

Node B’s GT 750M is Kepler `sm_30` — dropped after CUDA 10.2, and at ~ 30 GFLOPS FP64 (1:24 ratio) it would lose to its own CPU on this FP64-bound kernel (`16-ncu-divergence.tex`). `make GPU=0` therefore skips `nvcc` entirely: `kernel_stub.cpp` satisfies `kernel.h`’s three `extern "C"` symbols and fails loudly if `gpuEnable=1` is forced; `-DNO_NVTX` replaces the NVTX header with no-op inlines; the system `g++` compiles and links (the `g++-15` pin exists only for `nvcc` compatibility).

3. Methodology

Tabla 1: The two nodes. The per-frame throughputs in the last row are measured in $\mathcal{S}4$ (production spec, means); their ratio, $4.56\times$, is the single number that drives every benchmark result below.

	Node A (laptop)	Node B (ivy)
CPU	i7-9750H, 6C/12T (Coffee Lake)	i7-4702MQ, 4C/8T (Haswell)
GPU	GTX 1660 Ti Max-Q (CUDA)	GT 750M (<code>sm_30</code> , unusable)
Build	<code>make (g++-15 + nvcc)</code>	<code>make GPU=0 (g++ 16)</code>
Run config	hybrid 12 1 0.1 32768 1 1	CPU-only 8 0 0.1 32768 1 1
Solo wall (100 frames)	18.75 ± 0.34 s	85.48 s
Per-frame throughput	0.188 s/frame	0.855 s/frame

All runs use the production spec (100 frames, 1920×1080 , `diffT` = 0.1, `pixT` = 32768, `quiet` = 1, `save` = 1), both machines on AC (the benchmark driver refuses to start otherwise, per the repository’s battery-throttling methodology).

Identity checks (`verify_dist.sh`).

Byte-identity is only well-defined with the executor pinned: the CPU and CUDA `diverge()` differ in last-ulp FP64 rounding on deep-frame boundary pixels ($\sim 8\%$ of pixels GPU-vs-CPU, `20-igpu-opencl.tex`), and the hybrid’s region→executor assignment is timing-dependent — a pre-existing single-node property, not something distribution introduced. The checks therefore mirror experiments 21 (CPU-only `cmp`) and 23 (GPU-only): $N = 3$ simulated nodes on the laptop (hosts “:”, serialized) in three modes, and one real two-machine run, each compared frame-by-frame (`cmp`, exact bytes) against a single-process reference of the same binary and configuration.

Benchmark (`bench_2node.sh`).

Five timed configurations, 3 reps each: the laptop-alone hybrid baseline; four two-node runs — cyclic (block = 1), block-cyclic (block = 8), contiguous block (block = 50; ivy gets the deep frames 50–99), and the reversed block (hosts order swapped; ivy gets the cheap frames 0–49) — plus one ivy-solo capability rep. Two walls are recorded per run: `wall_e2e_s` times the whole orchestration (ship + ranks + PNG collection), and each node’s own `[total_elapsed_s]` isolates compute from plumbing.

4. Results

Tabla 2: Identity validation: distributed output vs. a single-process reference of the same binary and config, production spec. “Collected” is the union across ranks; every check must (and does) reach 100/100 on both columns. The last row is the real two-machine run — different compiler, different microarchitecture, different build flavour on node B.

Check	Nodes	Block	Collected	Byte-identical
CPU-only cyclic	3 (local)	1	100/100	100/100
GPU-only cyclic	3 (local)	1	100/100	100/100
CPU-only block-cyclic	3 (local)	8	100/100	100/100
CPU-only cyclic	2 (laptop + ivy)	1	100/100	100/100

Tabla 3: The heterogeneous benchmark: means $\pm \sigma$ over 3 reps. Δ compares end-to-end wall against the laptop-alone baseline — positive means the two-node configuration is *slower* than not distributing at all; every row is positive. “Overhead” is e2e – max(rank): spec shipping, SSH, and PNG collection.

Config	e2e (s)	laptop (s)	ivy (s)	Ovh. (s)	Δ
laptop alone (baseline)	18.75 $\pm$ 0.34	18.75	—	—	—
cyclic (block=1)	46.90 $\pm$ 1.07	10.33 $\pm$ 0.20	42.67 $\pm$ 1.27	4.2	+150%
block-cyclic (block=8)	45.00 $\pm$ 0.87	9.91 $\pm$ 0.09	41.54 $\pm$ 0.58	3.5	+140%
block (ivy deep half)	65.12 $\pm$ 1.70	5.70 $\pm$ 0.11	61.55 $\pm$ 1.47	3.6	+247%
block rev. (ivy cheap half)	28.24 $\pm$ 0.58	12.52 $\pm$ 0.11	24.86 $\pm$ 0.75	3.4	+51%
ivy alone (1 rep)	85.48	—	85.48	—	+356%

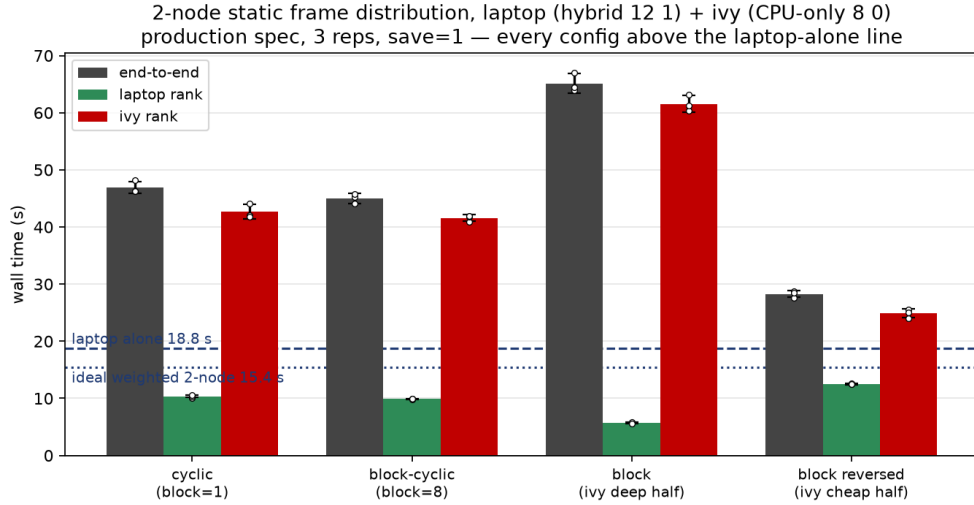


Figure 1: The four two-node configurations (bars: 3-rep means, whiskers σ , dots individual reps). Grey is the end-to-end wall, green/red the two ranks’ own compute walls. Every grey bar sits above the dashed laptop-alone line (18.8 s); even the dotted line — the analytic best a perfectly weighted static split could reach (15.4 s) — is only 18% below the baseline. The red bar is the binding rank in all four configurations.

5. Analysis

5.1. Distribution changes which frames exist, and provably nothing else

All four identity checks in Table 2 are 100/100 exact — 300 frame comparisons across process counts, assignment shapes, and, in the last row, physical machines. The reason is structural: the predicate gates only the `MandelFrame` allocation and the root-region enqueue, while the two inputs that determine pixel values — the accumulated corner interpolation and the `diverge()` arithmetic — run identically whether

or not a frame is owned. The cross-machine row is the strong version of the claim: node B computes with a different compiler (g++ 16 vs. g++-15), a different microarchitecture (Haswell vs. Coffee Lake), and a different build flavour (GPU=0 stub, -DNO_NVTX), and still reproduces every byte. This works because the build uses plain -O2 with no -march: both compilers emit baseline x86-64 scalar SSE2 FP64 with no FMA contraction, so the orbit recurrence rounds identically instruction by instruction. The implication cuts both ways: the identity guarantee is real, and it is *fragile* — adding -march=native to the CPU flags would break it through FMA contraction, the same last-ulp mechanism that already separates CPU from CUDA output.

5.2. Equal shares strand the wall on the slow node

The headline: cyclic — the assignment that *solves* the zoom’s cost trend — runs 46.90 s against 18.75 s alone, +150%. The mechanism is the throughput ratio in Table 1: ivy needs 0.855 s per production frame where the laptop hybrid needs 0.188 s ($4.56\times$), so any assignment that hands both nodes ~ 50 frames makes the wall $\max(10.3, 42.7) = 42.7$ s of compute plus overhead. The laptop finishes its share and idles for $\sim 76\%$ of ivy’s rank. Block-cyclic(8) is a wash with cyclic (45.00 vs. 46.90 s, 1.6σ apart): the interleave shape only redistributes cost *within* each node’s share, and both shares are the same ~ 50 frames. Interleave cannot fix share size — that is the defining limitation of the uniform predicate on non-uniform hardware.

5.3. Block orientation is a built-in 70/30 weighting — and measures the cost trend

The two contiguous-block runs bracket the whole benchmark: 65.12 s with ivy on the deep half, 28.24 s reversed — a $2.3\times$ swing from orientation alone. The halves are not equal in work: the deep half carries 68.7% of the laptop’s solo cost (12.52 vs. 5.70 s) and 71.2% of ivy’s (61.55 vs. 24.86 s). That two machines with different ISAs and executors agree on $\approx 70:30$ confirms the split is a property of the workload — the zoom’s cost trend integrated over each half — not of any node. The reversed block is therefore a crude weighted assignment (fast node takes the 70% half), and it is the best two-node configuration measured; but even this “right” orientation loses to the baseline by +51%, because a 70/30 work split is still far from the 82/18 split the throughput ratio demands.

5.4. The ceiling is analytic: $1.22\times$, and overhead eats it

No scheduler — static, weighted, or the future dispenser — can beat the combined throughput bound: $1/18.75 + 1/85.48 = 0.0650 \text{ s}^{-1}$, an ideal two-node wall of 15.4 s, a $1.22\times$ speed-up, with ivy carrying 18 frames. The measured orchestration overhead (3.4–4.2 s: spec shipping, SSH session setup, and ~ 50 deep-frame PNGs collected back over the LAN) consumes most of the ideal 3.4 s gain before scheduling quality even enters. The conclusion is not that distribution failed — the mechanism does exactly what it claims — but that this *pair* has nothing to distribute for latency: a node $4.56\times$ slower contributes at most 18% of the work, and the honest use of ivy is capacity (rendering a second spec, or rendering while the laptop does something else, where its 85 s/spec stands alone) rather than shortening the laptop’s 18.75 s.

A dispenser is *necessary* for heterogeneous pools — it would find the 82/18 split automatically — but on this pair it would be barely *sufficient*, chasing a sub-20% ideal through several seconds of fixed overhead.

5.5. What remains of the report-13 earmark

The block-vs-cyclic question this experiment was built to answer is deliberately unanswerable on heterogeneous hardware: when one rank binds at more than twice the other, the interleave’s effect (46.90 vs. 45.00 s) is noise against the share-size effect (28.24 vs. 65.12 s). What this pass *does* establish for the homogeneous study: the harness works end-to-end on real machines, per-rank compute is cleanly separable from the 3.4–4.2 s orchestration overhead, and the 70:30 half-split quantifies exactly the cost trend that cyclic assignment exists to neutralize. On N comparable nodes the prediction is direct: block strands the deep-block node at $\sim N \times 0.7\times$ the mean share (the 65.12 s row is this effect at $N = 2$), while cyclic and block-cyclic should tie within noise — block-cyclic(8) trading a slightly coarser mix for fewer cross-frame working-set switches.

5.6. A caveat on absolute walls: the baseline moved

Today’s laptop baseline is 18.75 ± 0.34 s where `23-gpu-periodicity.tex` recorded 23.72 s steady-state on the same code lineage — 21% faster, far outside both σ ’s. Nothing in this branch touches the render path (the identity checks prove it byte-exact), so the shift is environmental — thermal headroom and/or driver drift between measurement days. Every comparison in this report is same-day and unaffected; the lesson for cross-report arithmetic is that absolute walls need a same-day baseline, which is why Table 3 carries its own.

6. Recommendations

1. **Ship the mechanism; gate nothing on the benchmark.** The `DIST_*` predicate is 100/100 byte-identical in all four checks, costs nothing when unset, and cuts per-node frame-buffer memory to owned frames. The negative benchmark result is a property of this hardware pair, not of the code.
2. **Do not use ivy to shorten production walls.** Ceiling $1.22\times$ (15.4 s ideal vs. 18.75 s alone) minus 3.4–4.2 s overhead is a wash at best. Use ivy for capacity: a second spec in parallel is worth its full 85 s/spec.
3. **If a heterogeneous pool must cut latency, build the dispenser at the documented seam** (or add a weighted share to the predicate). Expected gain on this pair: $18.75 \rightarrow \sim 15.4$ s compute before overhead — measure the overhead first; it is currently half the prize.
4. **Run the earmarked block/cyclic/block-cyclic study on comparable nodes only** (S5.5); on heterogeneous hardware the share-size effect buries the interleave effect by an order of magnitude. Keep cyclic as the default meanwhile — it is never worse than block-cyclic here and neutralizes the measured 70:30 trend by construction.

5. **Never add `-march=native` to the CPU flags** while cross-machine byte-identity is part of the methodology ($\mathcal{S}5.1$); FMA contraction breaks last-ulp equality. If per-machine tuning is ever wanted, downgrade the identity check to the ± 1 -iteration equivalence used for CUDA-vs-OpenCL in `20-igpu-opencl.tex`.

7. Conclusion

The two-level hierarchy holds up exactly as designed: a one-line static ownership predicate distributes frames across machines with output byte-identical to a single-node run — 100/100 in every executor-pinned check, including across compilers, microarchitectures, and build flavours — because ownership gates allocation, never arithmetic. The first measured pass of the static-LB study, run on a deliberately heterogeneous pair, returns a clean negative: with a $4.56\times$ per-frame throughput gap, every equal-share configuration loses to the fast node alone (best +51%, worst +247%), block orientation swings $2.3\times$ by handing one node the 70% half of the zoom’s cost trend, and the analytic ceiling of $1.22\times$ — half of it consumed by 3.4–4.2 s of orchestration overhead — says no scheduler can rescue this pair for latency. The mechanism ships; the slow node earns its keep on capacity; the interleave comparison awaits homogeneous hardware, where the 65-second block row already previews the failure mode cyclic assignment exists to prevent.